IRMF with `pimf` versus AS222-style FFT

The supplied five-sine benchmark, reproduced with `pimf 0.4.0`.

Result

The package reproduces all 25 source-report accuracy entries within their displayed rounding. The FFT comparator still fails the agreement checkpoint in all 100 component pairs. At $n = 1000$, `pimf` takes **39.29 ms with 16 workers**, versus 12.99 ms with one worker and 0.231 ms for FFT. Using all cores slows this small problem; at $n = 8000$, it gives a **5.51-fold speedup**.

Same experiment, packaged local solver

Use $t_i = i/n$, $n = 1000$, and

$$Y_i = \sum_{j=1}^{5} w_j \sin(2\pi f_j t_i) + \varepsilon_i, \qquad \varepsilon_i \overset{\text{iid}}{\sim} N(0, 0.2^2).$$

Component	f_j	w_j	h_j
1	1	0.30	0.10000
2	3	0.25	0.08290
3	8	0.20	0.03030
4	20	0.15	0.01280
5	50	0.10	0.00757

20 paired realizations; seed 20261004; one validation draw first. Classical Epanechnikov weights, periodic boundaries, robustness $H = 1$. Each method subtracts its own component before the next bandwidth. No retuning or contamination is added.

`pimf` uses `IMF`, `SmoothAbs(1)`, and `Epanechnikov()`. One public `decompose(..., h1=h, k_max=1)` call per bandwidth preserves the nongeometric schedule and its exact fractional widths. Settings are `lookup=True`, `tol=1e-6`, `max_iter=60`, and `workers=16`.

Component agreement and true-sine recovery

Component / f	$r(P,F)$	RMS$(P-F)$	Relative RMS	$r(P,T)$	$r(F,T)$
1 / 1	0.9937	0.04001	16.8%	0.8562	0.7947
2 / 3	0.8006	0.02896	64.9%	0.9581	0.6657
3 / 8	0.9921	0.03105	27.0%	0.9727	0.9643
4 / 20	0.9464	0.02741	34.0%	0.8631	0.8890
5 / 50	0.9129	0.02330	49.2%	0.6997	0.7704

P: `pimf`; F: FFT; T: corresponding weighted true sine. Entries average metrics computed separately for each realization. Relative RMS is $\mathrm{RMS}(P-F)/\mathrm{RMS}(P)$. The inherited checkpoint requires $r(P,F) \geq 0.995$ and relative RMS $\leq 5\%$ in *every* pair. It fails, so the conditional 1,000-run extension is not triggered.

Why changing the implementation does not close the gap

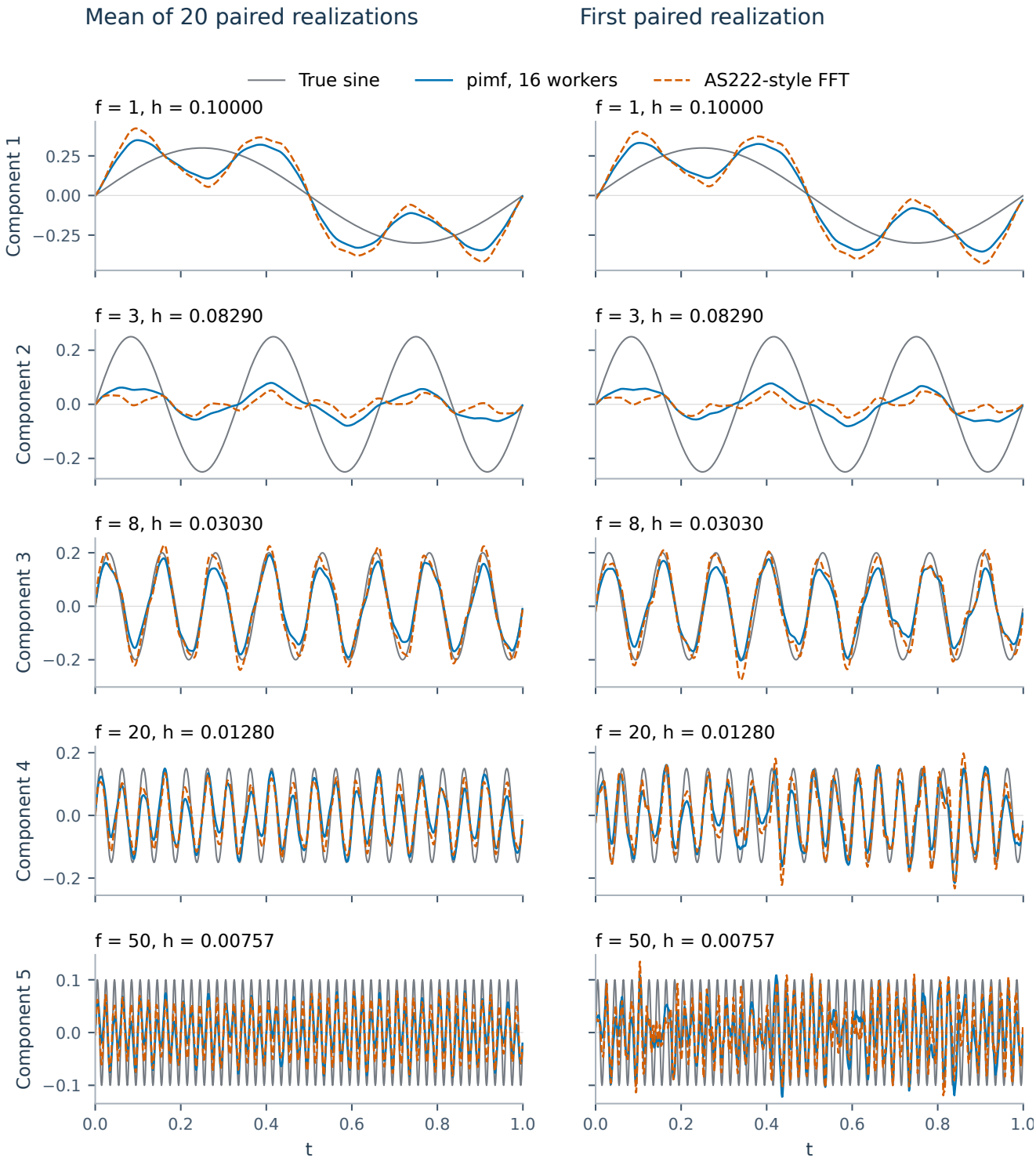
Let K be the common discrete kernel smoother and $\psi_H(u) = \mathrm{erf}(u/(\sqrt{2}H))$. Direct IRMF solves $\sum_i q_{ti}\psi_H(R_i - x_t) = 0$. The comparator instead uses

$$p = KR, \qquad S = p + \frac{K\psi_H(R-p)}{K\psi'_H(R-p)}.$$

Its scores are centered at observation-specific pilots p_i. For a linear score, it reduces to $(2K - K^2)R$, whereas direct smoothing gives KR. The discrepancy is in the estimator; FFT arithmetic is accurate.

Recovered components

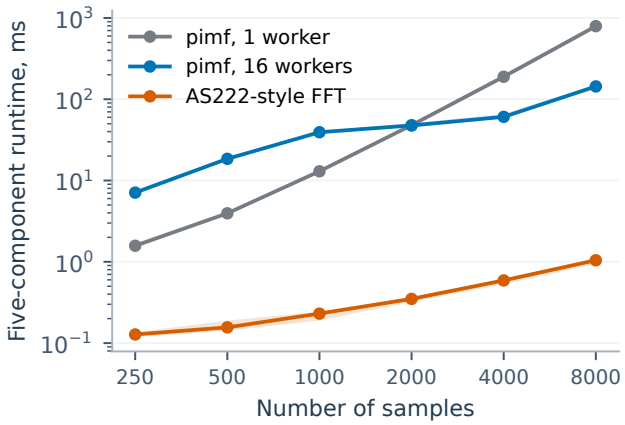
Same observations and bandwidths; each method uses its own residual recursion.



Component 2 has the largest relative disagreement, 64.9% on average. The mean curves expose persistent attenuation and leakage; the single-run curves show noise variation. The correlations on page 1 are per-realization averages, not correlations of the mean curves.

Runtime, checks, and original sources

Fresh timing measurements



n	1 worker	16 workers	FFT	Speedup
250	1.58	7.10	0.128	0.22×
500	3.95	18.50	0.156	0.21×
1,000	12.99	39.29	0.231	0.33×
2,000	47.94	47.61	0.350	1.01×
4,000	189.15	60.75	0.590	3.11×
8,000	793.37	143.88	1.045	5.51×

Times in ms; speedup = 1-worker / 16-worker time.

Five warmed repetitions per size, with all three configurations retimed in randomized order. Timing includes five smoothing calls, thread-pool creation, and residual updates. Kernel weights and FFT spectra are precomputed; simulation, setup, checks, and plotting are excluded. Shading shows the observed range. Cached and uncached package outputs are bit-identical.

Apple M4 Max, 16 cores; Python 3.14.6; NumPy 2.5.4; SciPy 1.18.1. Numerical-library threads remain at one. `workers=16` gives pool capacities of 4, 8, and 16 for $n = 250$, 500, and $n \geq 1000$, because `pimf` requires at least 64 samples per chunk. Parallel overhead outweighs gains at small n; all measured FFT times remain lower.

Numerical checks

Check	Maximum absolute error
16-worker `pimf` versus 25 exact-score Brent roots	1.28×10^{-7}
FFT versus spatial evaluation of the same one-step formula	2.22×10^{-16}
Package versus independently sampled kernel weights	2.08×10^{-17}
Default package versus tighter solver with score lookup disabled	1.69×10^{-7}
Reconstruction across all 20 paired runs	4.44×10^{-16}
One versus 16 workers across all 20 paired runs	1.34×10^{-7}

All 100 parallel package stages converged, with at most 7 iterations. Thread chunks stop independently, explaining small differences near solver tolerance. The root and tight-solver checks use the separate validation realization.

What the original papers establish

Härdle's AS222 paper is from **1987**; the **1988** paper is Dusoir's correction. Equation 2 of AS222 gives the pilot-plus-score-ratio construction, using Huber scores and Gaussian smoothing. Page 106 explicitly notes the changed score centers. Dusoir corrects initialization of `WK(5:6,:)` and `M1=M/2`. Both full publisher PDFs were retrieved and checked.

This experiment preserves the supplied report's Gaussian-score, Epanechnikov-kernel adaptation, with an unbinned periodic grid. It does not reproduce historical Fortran outputs or the original contaminated random-design example. Modern kernel transforms are explicitly initialized and inverse FFTs specify their length. Twenty Gaussian-noise realizations establish this comparison only; they do not test resistance to outliers or automatic bandwidth selection.

Reproduce and inspect

The experiment README at `experiments/pimf-as222/README.md` gives the commands and pinned dependencies. Raw paired metrics, timing repetitions, curves, solver diagnostics, checks, and source hashes are in `output/pimf-as222/`.

Sources. Supplied comparison PDFs dated 4 and 8 October 2026. W. Härdle, *Resistant Smoothing Using the Fast Fourier Transform*, 1987, 36:104–111, doi:10.2307/2347850. T. Dusoir, *Remark AS R73*, 1988, 37:316–317, doi:10.2307/2347358. pimf 0.4.0, PyPI. The source reports' code and saved arrays were not supplied; matching their rounded tables does not establish bitwise reproduction of their curves.